



CAHIER DES CHARGES

FONCTIONNEL ET TECHNIQUE - VERSION SYNTHÉTIQUE

ChatOps AI

Automatisation DevOps de bout en bout pour un chatbot IA évolutif

Document

Version condensée à 14 pages, regroupant les exigences essentielles, les critères de recette et les choix d'architecture.

Champ	Valeur
Auteur	Lan Evrad DSOH TCHAMEN
Titre visé	Administrateur système DevOps
Organisme	École-IT
Version	1.1 - Juin 2026
Statut	Document de référence pour le projet et la soutenance

0. Identification et sommaire

Cadre documentaire, priorités et structure de lecture

Fiche d'identification

Élément	Description
Nom du projet	ChatOps AI
Nature	Plateforme conversationnelle IA avec chaîne DevOps complète
Technologies principales	Next.js, FastAPI, PostgreSQL, Docker, GitHub Actions, Terraform, AWS, Prometheus/Grafana
Public visé	Utilisateurs, équipes DevOps, administrateurs et jury
Environnement cible	Prototype AWS EC2 ; évolution prévue vers Kubernetes

Priorisation des exigences

Niveau	Interprétation
Must	Indispensable à la validation de la version initiale.
Should	Important, mais reportable sans bloquer le cœur fonctionnel.
Could	Amélioration utile si le temps et les ressources le permettent.
Won't / Roadmap	Hors version initiale, documenté pour une évolution ultérieure.

Sommaire synthétique

Page	Contenu
3	Contexte, problématique, objectifs et résultats attendus
4	Acteurs, rôles, périmètre, hypothèses et contraintes
5	Parcours utilisateurs et exigences fonctionnelles
6	Données, API, incidents et règles de gestion
7	Architecture technique et composants
8	Infrastructure AWS, Terraform et sécurité
9	CI/CD, Docker et Kubernetes
10	Supervision, logs et incidents
11	Exigences non fonctionnelles et niveaux de service
12	Tests, recette, déploiement et continuité
13	Planning, risques et livrables
14	Critères de réussite, compétences, roadmap et conclusion

1. Présentation générale et enjeux

Pourquoi ChatOps AI et quels résultats sont attendus ?

1.1 Contexte

Les équipes numériques doivent livrer rapidement tout en garantissant sécurité, disponibilité et observabilité. Les interfaces conversationnelles basées sur l'IA facilitent l'accès à l'information, tandis que les pratiques DevOps automatisent la construction, les tests, le déploiement et l'exploitation.

ChatOps AI réunit ces deux dimensions : une application web permet à un utilisateur authentifié de dialoguer avec une IA, de consulter son historique et, selon son rôle, d'accéder à des fonctions d'exploitation. L'ensemble est conteneurisé, déployé automatiquement et supervisé.

1.2 Problématique

Question centrale	Comment concevoir, déployer et exploiter une plateforme conversationnelle intelligente qui soit reproductible, sécurisée, observable et capable d'évoluer vers une architecture orchestrée ?
--------------------------	--

1.3 Objectifs

Fournir une interface moderne pour l'inscription, la connexion, le chat et l'historique.
Protéger les accès par JWT et contrôler les actions avec les rôles viewer, devops et admin.
Persister utilisateurs, conversations, incidents et journaux d'audit dans PostgreSQL.
Conteneuriser les services et garantir un démarrage reproductible avec Docker Compose.
Automatiser tests, build et déploiement sur AWS EC2 via GitHub Actions.
Provisionner l'infrastructure par Terraform et centraliser métriques, logs et alertes.
Préparer une évolution vers Kubernetes, la haute disponibilité et l'auto-scaling.

1.4 Résultats et preuves attendus

Résultat	Preuve
Plateforme accessible	Frontend, API, Swagger et endpoint /health disponibles.
Conversation sécurisée	Un utilisateur connecté obtient une réponse IA ; l'échange est lié à son compte.
Déploiement reproductible	docker compose up démarre les composants sans installation manuelle.
Livraison automatisée	Le pipeline exécute tests, build et déploiement après validation.
Observabilité	Prometheus collecte les métriques ; Grafana les affiche ; des alertes sont définies.
Infrastructure versionnée	Terraform décrit le réseau, la sécurité et le serveur AWS.

2. Acteurs, rôles et périmètre

Responsabilités, droits d'accès, inclusions et limites

2.1 Parties prenantes

Acteur	Responsabilités / attentes
Porteur du projet	Conception, développement, infrastructure, documentation, démonstration et maintenance.
Utilisateur	Dialoguer avec l'IA et consulter uniquement son propre historique.
Équipe DevOps	Consulter l'état des services, gérer les incidents et diagnostiquer les anomalies.
Administrateur	Gérer les rôles et exécuter les actions sensibles avec traçabilité.
Formateur / jury	Évaluer les choix, les preuves, les limites et la maîtrise du référentiel.

2.2 Profils et droits

Fonction	Viewer	DevOps	Admin
Chat et historique	Oui	Oui	Oui
Consulter métriques / état	Lecture	Oui	Oui
Gérer les incidents	Non	Oui	Oui
Consulter pods et logs	Non	Oui	Oui
Redémarrer un déploiement	Non	Non	Oui
Gérer les rôles	Non	Non	Oui

2.3 Périmètre inclus et exclusions

Inclus dans la version initiale	Hors périmètre initial
Frontend, API REST, authentification, chat IA et historique	Entraînement d'un modèle IA propriétaire
PostgreSQL, Docker Compose, tests et documentation Swagger	Application mobile native et facturation
Terraform AWS, GitHub Actions et supervision	Haute disponibilité multi-région
Incidents, audit, rôles et intégration Kubernetes préparée	Cluster Kubernetes managé totalement industrialisé
Prometheus, Grafana, Alertmanager, Loki/Promtail	Support 24 h/24 avec astreinte contractuelle

2.4 Hypothèses et contraintes

Accès Internet disponible pour OpenAI, GitHub et AWS ; clés fournies par les responsables autorisés.

Prototype hébergé sur une instance EC2 à budget limité ; charge attendue modérée.

Secrets exclus du dépôt et injectés par variables d'environnement ou GitHub Secrets.

Utilisation prioritaire de composants open source ; dépendance externe à OpenAI gérée par timeouts et messages contrôlés.

HTTPS et restriction réseau obligatoires pour une mise en production publique.

3. Parcours et exigences fonctionnelles

Fonctions indispensables et critères d'acceptation

3.1 Parcours nominal

L'utilisateur crée un compte ou se connecte ; le backend valide les données et émet un JWT limité dans le temps.
 Le frontend transmet le jeton dans l'en-tête Authorization pour chaque route protégée.
 L'utilisateur envoie un prompt non vide ; le backend contrôle son identité, appelle OpenAI et enregistre l'échange.
 L'historique est filtré par user_id afin qu'un utilisateur ne voie que ses propres conversations.
 Les utilisateurs DevOps/Admin accèdent aux incidents et informations techniques selon leurs droits.

3.2 Exigences principales

ID	Fonction / exigence	Priorité	Critère d'acceptation
FR-01	Inscription avec nom, e-mail et mot de passe ; rôle viewer par défaut.	Must	Compte créé, mot de passe illisible en base.
FR-02	Connexion et émission d'un JWT contenant identité et rôle.	Must	Token valide ; erreur 401 si identifiants invalides.
FR-03	Protection des routes et contrôle des rôles.	Must	Accès refusé sans token ou avec rôle insuffisant.
FR-04	Déconnexion côté client et suppression de la session locale.	Must	Token retiré et redirection vers la connexion.
FR-05	Envoi d'un prompt authentifié vers le service IA.	Must	Réponse 200 et contenu affiché.
FR-06	Persistance de chaque échange avec user_id et date.	Must	Conversation enregistrée et rattachée au compte.
FR-07	Consultation de l'historique personnel.	Must	Aucune donnée appartenant à un autre utilisateur.
FR-08	Gestion contrôlée des erreurs OpenAI et réseau.	Should	Message utilisateur clair, sans fuite technique.
FR-09	Documentation interactive des routes REST.	Should	Swagger accessible sur /docs.
FR-10	Endpoint de santé et métriques.	Must	/health et /metrics répondent correctement.

3.3 Erreurs attendues

Robustesse	Token absent ou expiré : 401 ; rôle insuffisant : 403 ; prompt vide : 422 ; service IA indisponible : message contrôlé ; base indisponible : erreur journalisée et endpoint /health dégradé.
-------------------	--

4. Données, API et exploitation

Modèle de données, routes REST et règles de gestion

4.1 Modèle de données

Entité	Champs principaux	Règles
users	id, username, email, password_hash, role, created_at	Nom/e-mail uniques ; mot de passe haché ; rôle contrôlé.
conversations	id, user_id, user_message, ai_response, created_at	user_id obligatoire ; isolation stricte des historiques.
incidents	id, title, description, severity, status, created_by, dates	Sévérité et statut limités à des valeurs autorisées.
audit_logs	id, user_id, action, target, details, created_at	Actions sensibles horodatées et non modifiables par un viewer.

4.2 Routes REST essentielles

Méthode	Route	Accès	Finalité
POST	/register	Public	Créer un compte.
POST	/login	Public	Authentifier et retourner un JWT.
POST	/chat	Authentifié	Envoyer un prompt et enregistrer la réponse.
GET	/history	Authentifié	Retourner l'historique de l'utilisateur.
GET	/health	Public/technique	Vérifier l'état de l'API et de la base.
GET	/metrics	Supervision	Exposer les métriques Prometheus.
GET/POST/ PATCH/ DELETE	/incidents	DevOps/Admin	Gérer le cycle de vie d'un incident.
GET	/k8s/pods, /deployments, /services	DevOps/Admin	Consulter l'état du cluster.
POST	/k8s/restart/{name}	Admin	Redémarrer un déploiement autorisé.
GET	/audit	Admin	Consulter les actions sensibles.

4.3 Règles de gestion

Le mot de passe n'est jamais retourné par l'API et ne doit apparaître dans aucun journal.

Un utilisateur ne peut lire ni modifier les données d'un autre compte.

Les opérations d'administration exigent une autorisation explicite et génèrent une trace d'audit.

Les suppressions d'incidents sont réservées à l'administrateur ; les transitions de statut sont contrôlées.

Les erreurs internes sont journalisées côté serveur, mais les réponses publiques restent génériques.

5. Architecture technique

Découpage logique, technologies et principes de conception

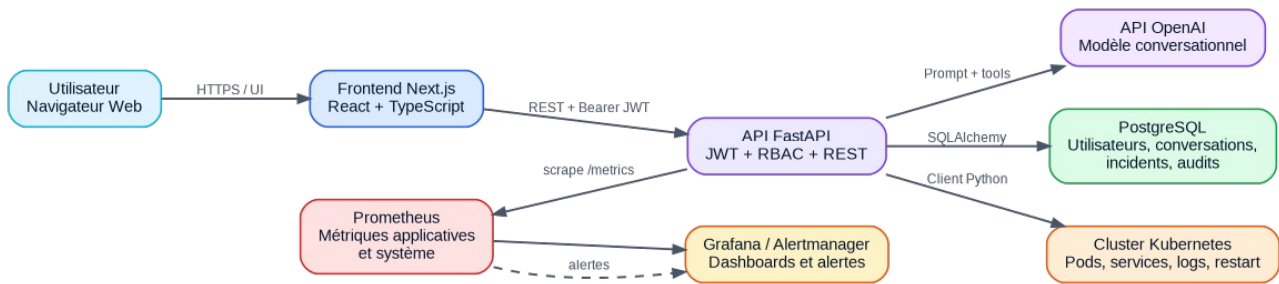


Figure 1 - Architecture logique de ChatOps AI.

5.1 Composants

Couche	Technologies	Responsabilités
Présentation	Next.js, React, TypeScript	Pages, session utilisateur, chat, historique et dashboard.
API	Python, FastAPI, Pydantic	Validation, logique métier, IA, RBAC, incidents, audit et endpoints techniques.
Données	PostgreSQL, SQLAlchemy	Persistance transactionnelle et relations entre utilisateurs et conversations.
IA	API OpenAI	Génération de réponses avec gestion des erreurs et quotas.
Conteneurs	Docker, Docker Compose	Isolation, réseau interne, volumes et démarrage reproductible.
Cloud / IaC	AWS EC2, VPC, Terraform	Hébergement, réseau, sécurité et versionnement de l'infrastructure.
CI/CD	GitHub Actions	Tests, build, transfert et déploiement automatisés.
Observabilité	Prometheus, Grafana, Alertmanager, Loki	Métriques, tableaux de bord, alertes et centralisation des logs.
Orchestration	Kubernetes - préparation	Déploiements, services, réplication et fonctions d'exploitation futures.

5.2 Principes

Séparation claire des responsabilités entre interface, API, données et exploitation.

Configuration externalisée par environnement ; aucune clé sensible versionnée.

Services stateless autant que possible ; persistance assurée par PostgreSQL et volumes.

Health checks, métriques et logs intégrés dès la conception.

Architecture évolutive : Docker Compose pour le prototype, Kubernetes pour l'industrialisation.

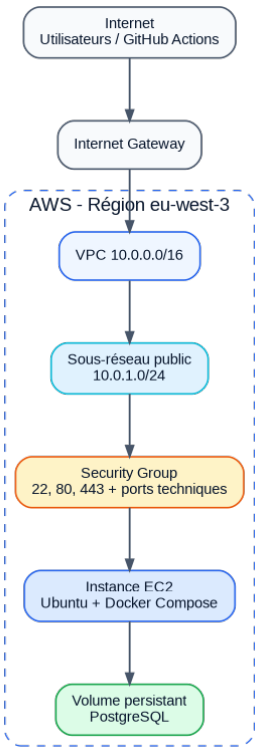
6. Infrastructure AWS et sécurité

Provisionnement Terraform et réduction de la surface d'exposition



6.1 Ressources cibles

- Région eu-west-3 (Paris) et VPC 10.0.0.0/16 avec DNS.
- Sous-réseau public, Internet Gateway et table de routage.
- Instance EC2 Ubuntu t3.micro pour le prototype.
- Groupe de sécurité limitant les flux entrants aux ports nécessaires.
- Terraform pour init/plan/apply, variables et outputs.
- Adresse IP publique fournie en sortie ; accès cible via domaine et HTTPS.



6.2 Exigences de sécurité

ID	Exigence	Priorité
SEC-01	Clés, mots de passe et tokens externalisés et masqués dans les logs.	Must
SEC-02	Accès public par HTTPS avec certificat valide.	Must
SEC-03	SSH restreint ou remplacé par AWS Systems Manager.	Should
SEC-04	Prometheus, Grafana et PostgreSQL non exposés directement à Internet.	Must
SEC-05	Principe du moindre privilège pour les rôles IAM et les comptes applicatifs.	Must
SEC-06	Mises à jour, scan d'images et rotation des secrets documentés.	Should

6.3 Écarts du prototype et remédiation

Point de vigilance	Certains ports ont été ouverts largement pour les essais. Avant production : fermer les ports techniques, utiliser un reverse proxy Nginx, activer TLS, limiter les Security Groups, préférer SSM à SSH et isoler la base sur un réseau privé.
--------------------	--

7. CI/CD, Docker et Kubernetes

Automatisation de la livraison et préparation de l'orchestration

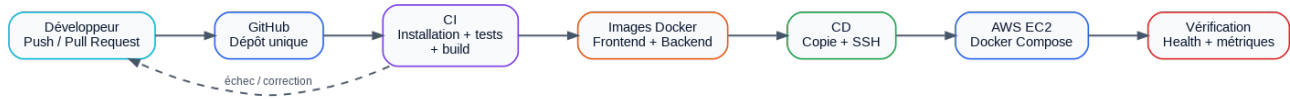


Figure 2 - Pipeline de livraison automatisée.

7.1 Pipeline GitHub Actions

Étape	Action	Condition de réussite
Checkout	Récupérer le commit à livrer.	Sources accessibles et version identifiée.
Tests backend	Installer Python et lancer Pytest.	Tous les tests obligatoires sont verts.
Build	Construire les images frontend et backend.	Images créées sans erreur.
Sécurité	Scanner dépendances et images avec Trivy.	Aucune vulnérabilité critique non acceptée.
Déploiement	Transférer les fichiers et exécuter Docker Compose sur EC2.	Services démarrés et réseau disponible.
Contrôle	Interroger /health et collecter les logs en cas d'échec.	Health check réussi dans le délai.
Rollback	Redémarrer la dernière version stable si nécessaire.	Service rétabli et incident tracé.

7.2 Exigences Docker

Un Dockerfile par application ; images légères, utilisateur non-root et versions épinglées.

Docker Compose définit frontend, backend, PostgreSQL, volumes, réseau et health checks.

La base n'est pas exposée publiquement ; les services communiquent par leur nom DNS interne.

Les données persistent dans un volume et le fichier .env reste hors Git.

7.3 Kubernetes - évolution contrôlée

Les manifestes Deployment et Service ainsi que les fonctions de consultation des pods, services, déploiements et logs préparent l'industrialisation. La version initiale reste validée sur Docker Compose ; le passage Kubernetes exige un cluster opérationnel, un kubeconfig ou ServiceAccount, des Secrets, des probes, un Ingress TLS et des politiques RBAC.

8. Supervision, logs et incidents

Détecter rapidement, diagnostiquer et rétablir le service

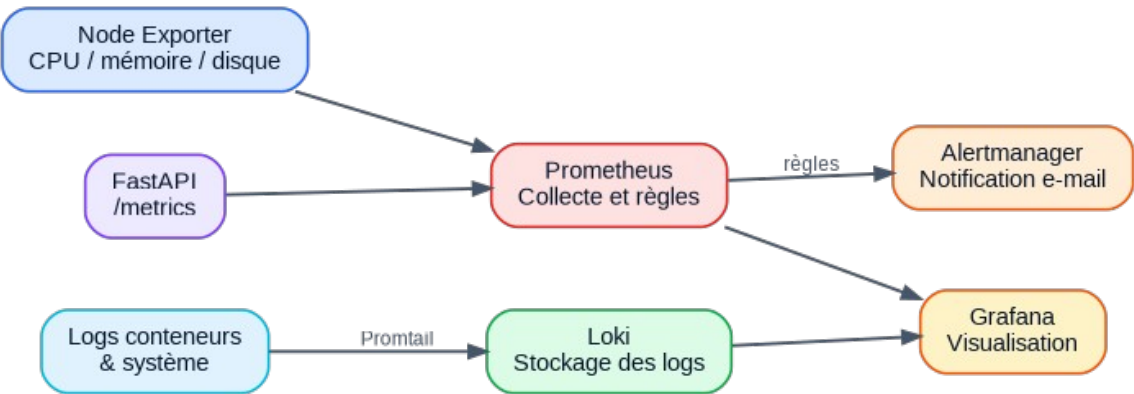


Figure 3 - Chaîne d’observabilité et d’alerte.

8.1 Métriques minimales

Catégorie	Indicateurs
Disponibilité	up des cibles, statut /health, uptime des conteneurs.
API	Requêtes, codes HTTP, latence, erreurs 5xx.
Métier	Nombre de conversations, échecs IA et erreurs de persistance.
Système	CPU, mémoire, disque, charge et réseau via Node Exporter.
Base	Disponibilité, connexions, taille et erreurs de requêtes.
Déploiement	Version active, date du dernier déploiement et statut du pipeline.

8.2 Alertes

Alerte	Condition indicative	Sévérité
BackendDown	Cible FastAPI indisponible plus d’une minute.	Critique
HighErrorRate	Taux de réponses 5xx supérieur au seuil pendant 5 minutes.	Critique
HighCPUUsage	CPU supérieur à 80 % pendant 2 minutes.	Avertissement
HighMemoryUsage	Mémoire supérieure à 85 % pendant 2 minutes.	Avertissement
LowDiskSpace	Espace libre inférieur à 15 %.	Avertissement
DatabaseUnavailable	Échec du contrôle de connexion PostgreSQL.	Critique

8.3 Logs et processus d’incident

Promtail collecte les logs structurés ; Loki les stocke ; Grafana permet la recherche corrélée.
Les secrets et données personnelles sont masqués ; les événements comportent date, niveau, service et identifiant de corrélation.
Processus : détection → qualification → mitigation → résolution → validation → clôture et retour d’expérience.
Chaque incident comprend titre, gravité, statut, responsable, actions et horodatage.

9. Exigences non fonctionnelles

Performance, disponibilité, sécurité, qualité et exploitabilité

ID	Domaine	Exigence / cible	Vérification
NFR-01	Disponibilité	Objectif prototype $\geq 99\%$ hors maintenance.	Prometheus et historique des alertes.
NFR-02	Performance API	P95 hors appel IA < 500 ms en charge nominale.	Test de charge et métriques.
NFR-03	Réponse IA	Timeout contrôlé ; message dégradé en cas d'indisponibilité.	Test de panne simulée.
NFR-04	Sécurité	Mots de passe hachés, JWT expirant, RBAC et secrets externalisés.	Tests sécurité et inspection configuration.
NFR-05	Confidentialité	Historique isolé par utilisateur et logs sans secrets.	Tests d'accès croisé.
NFR-06	Portabilité	Démarrage reproductible sur Linux avec Docker Compose.	Installation sur environnement vierge.
NFR-07	Maintenabilité	Architecture modulaire, README, Swagger et configuration documentée.	Revue du dépôt.
NFR-08	Observabilité	Métriques, logs, health checks et alertes disponibles.	Démonstration Grafana/Prometheus.
NFR-09	Sauvegarde	Sauvegarde régulière et restauration testée.	Journal de restauration.
NFR-10	Accessibilité	Interface responsive, navigation clavier et messages compréhensibles.	Recette fonctionnelle.
NFR-11	Scalabilité	Services stateless prêts à être répliqués.	Validation de l'architecture.
NFR-12	Traçabilité	Version du code, pipeline et actions d'administration identifiables.	Git, CI et audit logs.

9.1 Niveaux de service indicatifs

Indicateur	Cible
Détection d'un backend indisponible	≤ 2 minutes
Rétablissement d'un incident critique de démonstration	≤ 30 minutes
Perte maximale de données visée	≤ 24 heures avec sauvegarde quotidienne
Temps de restauration du prototype	≤ 2 heures
Conservation des logs techniques	30 jours, ajustable selon capacité
Principe	Ces objectifs sont adaptés à un prototype pédagogique. Une production commerciale nécessiterait dimensionnement, redondance, tests de charge et engagements contractuels.

10. Tests, recette et mise en service

Vérifier avant livraison et garantir la continuité minimale

10.1 Niveaux de tests

Niveau	Objet	Outils / exemples
Unitaire	Sécurité, validation et services isolés.	Pytest, mocks et tests de rôles.
API	Routes, codes HTTP, schémas et JWT.	FastAPI TestClient.
Intégration	API + PostgreSQL + migrations + OpenAI simulé.	Docker Compose et base de test.
Frontend	Connexion, chat, erreurs et navigation.	Tests composants / parcours.
Infrastructure	Syntaxe et plan Terraform.	terraform fmt, validate et plan.
Sécurité	Secrets, dépendances, images et accès.	Trivy, revue CORS/RBAC, scan Git.
Recette	Parcours métier et exploitation.	Scénarios documentés et preuves.

10.2 Scénarios de recette essentiels

ID	Scénario	Résultat attendu
REC-01	Inscription valide	Compte viewer créé ; mot de passe haché.
REC-02	Connexion invalide	401, sans information sensible.
REC-03	Chat authentifié	Réponse IA affichée et échange persisté.
REC-04	Accès croisé à l'historique	Aucune conversation d'un autre utilisateur.
REC-05	Action admin par viewer	Accès refusé avec code 403.
REC-06	Redémarrage des conteneurs	Données PostgreSQL conservées.
REC-07	Déploiement CI	Tests et health check réussis.
REC-08	Backend arrêté	Alerte déclenchée et incident qualifié.
REC-09	Restauration base	Sauvegarde restaurée et données vérifiées.

10.3 Mise en service et continuité

Préparer l'EC2, Docker, les variables, les secrets et le DNS ; lancer le pipeline ; vérifier /health, /docs et les tableaux de bord.

Sauvegarde quotidienne PostgreSQL, conservation chiffrée et test périodique de restauration.

Maintenance : mises à jour mensuelles, rotation des secrets, nettoyage Docker, revue des alertes et contrôle de l'espace disque.

Critère de sortie : toutes les exigences Must validées ou dérogation écrite, aucun secret dans Git et procédure de retour arrière disponible.

11. Planification, risques et livrables

Organisation du projet et mesures de maîtrise

11.1 Planning indicatif

Phase	Période	Livrables
Cadrage	S1	Besoins, périmètre, architecture et backlog.
Socle applicatif	S2-S3	Backend, frontend, PostgreSQL et authentification.
Fonctions métier	S4	Chat, historique, rôles, incidents et audit.
Conteneurisation	S5	Dockerfiles, Compose, volumes et health checks.
Cloud / IaC	S6	Terraform, VPC, Security Group et EC2.
CI/CD	S7	Tests, build, déploiement et contrôle post-déploiement.
Observabilité	S8	Prometheus, Grafana, alertes et logs.
Recette / dossier	S9	Tests finaux, preuves, documentation et soutenance.

11.2 Principaux risques

Risque	Impact	Réponse
Secret exposé dans Git	Élevé	Rotation immédiate, nettoyage, scan et secrets manager.
Indisponibilité OpenAI	Moyen	Timeout, retry limité et message dégradé.
Espace disque EC2 saturé	Élevé	Alertes disque, nettoyage images/logs et quotas.
Build NPM/Python instable	Moyen	Versions épinglées, cache contrôlé et retry.
Base indisponible au démarrage	Élevé	Health check, depends_on et mécanisme de retry.
Configuration Kubernetes absente	Moyen	Mode dégradé, message clair et validation kubeconfig.
Port technique exposé	Élevé	Fermeture, réseau privé, VPN/SSM et reverse proxy.
Déploiement défectueux	Élevé	Health check post-déploiement et rollback automatique.

11.3 Livrables

Livrable	Contenu
Code source	Frontend, backend, tests et configurations.
Infrastructure	Terraform, variables, outputs et guide AWS.
Conteneurs	Dockerfiles, Compose et volumes.
CI/CD	Workflows GitHub Actions et secrets requis.
Observabilité	Prometheus, Grafana, alertes, Loki/Promtail.
Documentation	README, Swagger, cahier des charges, dossier professionnel et support oral.
Preuves	Captures, logs, résultats de tests et démonstration.

12. Validation, compétences et roadmap

Conditions de réussite et trajectoire d'industrialisation

12.1 Critères globaux de réussite

Critère	Preuve attendue
Authentification opérationnelle	Inscription, connexion JWT et refus des accès non autorisés.
Chat et persistance	Réponse IA et conversation enregistrée avec user_id.
Isolation des données	Historique personnel uniquement.
Déploiement reproductible	Docker Compose et pipeline GitHub Actions fonctionnels.
Infrastructure automatisée	Terraform plan/apply et ressources AWS visibles.
Supervision opérationnelle	Métriques, dashboard et alerte sur indisponibilité.
Sécurité minimale	Aucun secret versionné, rôles appliqués et ports réduits.
Documentation complète	README, Swagger, procédures et preuves de recette.

12.2 Correspondance avec les compétences

Activité	Compétences démontrées	Éléments du projet
AT1 - Infrastructure cloud	CP1 à CP4	Terraform, VPC, EC2, Security Groups, secrets et mise en production AWS.
AT2 - Déploiement continu	CP5 à CP8	Pytest, PostgreSQL, Docker, GitHub Actions et préparation Kubernetes.
AT3 - Supervision	CP9 à CP11	Métriques, Prometheus/Grafana, logs, alertes, documentation et veille en anglais.

12.3 Roadmap

Horizon	Évolutions
Court terme	Consolider les workflows, ajouter Alembic, scan Trivy, health checks PostgreSQL et TLS Nginx.
Moyen terme	Cluster Kubernetes managé, Ingress, Secrets, HPA, registre d'images et base PostgreSQL managée.
Long terme	OpenTelemetry, haute disponibilité, sauvegardes externalisées, multi-environnements et IA locale/open source.

Conclusion

La version initiale de ChatOps AI est recevable lorsque les exigences Must sont démontrées, que la sécurité minimale est appliquée et que les écarts restants sont explicitement tracés. Ce cahier des charges synthétique constitue la référence commune pour développer, tester, déployer, superviser et faire évoluer la plateforme.

Décision de validation	Validation par le porteur du projet et le formateur après recette fonctionnelle, contrôle de l'infrastructure, vérification de la chaîne CI/CD et examen des preuves de supervision.
-------------------------------	--